

Project NEXA — Agent Runtime Harness Comparison

1. Common tool/skill calls (all three scenarios)

All three scenarios process the same CONTACTS / SGSS maintenance case and use the same set of tools. The mock OpenAPI schema for Bedrock AgentCore Gateway defines them as:

Tool / path	Purpose
classify_intent	Identifies the case as CONTACTS, HIGH confidence, SGSS work group
extract_fields	Pulls date of birth, address, tax id from the document
retrieve_sop	Returns required validation checks (address, tax_id, duplicate)
run_validations	Runs address/tax-id/duplicate checks
check_similar_cases / search_similar_cases	Searches Postgres+pgvector for similar prior cases
create_decision_task	Creates a human-approval task in the queue
execute_transaction	Posts the final approved transaction to Pega/SaliPoint
write_audit_record	Persists the audit log

2. How the harness works in each scenario

A. Claude Agent SDK on Bedrock AgentCore Runtime

- **Your code:** Claude Agent SDK packaged as an ARM64 container.
- **Runtime:** Bedrock AgentCore Runtime runs the container.
- **Invocation:** EventBridge invokes the Runtime directly.
- **Sessions:** Two distinct Runtime sessions – one before the human decision, one after.
- **Tool calls:** Runtime calls REST APIs through **Bedrock AgentCore Gateway** -> DataPower (mTLS/OAuth) -> on-prem REST endpoints.
- **State:** Session 2 re-reads state from Aurora and EventBridge instead of relying only on memory.

- **Key call flow:** `classify_intent -> extract_fields -> retrieve_sop -> run_validations -> check_similar_cases -> create_decision_task -> (human decides) -> execute_transaction -> write_audit_record.`

B. Strands Agents on Bedrock AgentCore Harness

- **Your code:** None / configuration only. Strands is the managed agent loop.
- **Runtime:** Bedrock AgentCore **Harness** (managed, configuration-driven loop) instead of user-provided Runtime code.
- **Invocation:** Same Gateway/REST targets as above.
- **Tool calls:** Harness calls the same nine skills through AgentCore Gateway.
- **Difference from Claude:** The loop itself is managed by the Harness service; you configure tools and the model, not the orchestration code.

C. LangGraph + AWS Step Functions

- **Your code:** `langgraph_lambda.py` plus a Step Functions JSON template.
- **Runtime:** AWS Step Functions standard state machine with `waitForTaskToken`.
- **States:** Intake -> human decision -> Execute.
- **Tool calls:** The intake Lambda directly invokes the `data_power_mock` Lambda for each tool; the state machine itself only orchestrates the two high-level steps.
- **State:** DynamoDB `nexa-langgraph-checkpoints` stores state and the `waitForTaskToken` token.
- **Decision boundary:** Step Functions `waitForTaskToken` in the Intake state; an external caller (`test_langgraph.py` or a human) sends `SendTaskSuccess` with `Approved/Denied`.

3. Comparison of runs

	Claude AgentCore Runtime	Strands AgentCore Harness	LangGraph + Step Functions
Agent loop owner	Your Claude SDK code	Bedrock AgentCore Harness service	<code>langgraph_lambda.py</code> inside intake Lambda
AWS orchestrator	Bedrock AgentCore Runtime	Bedrock AgentCore Harness	AWS Step Functions
Human decision point	Runtime session 1 ends -> queue task -> session 2 starts	Same, managed by Harness	Step Functions <code>waitForTaskToken</code> Intake and Execute
State storage	Aurora + EventBridge	Harness-managed	DynamoDB
Tool transport	AgentCore Gateway -> REST (DataPower)	AgentCore Gateway -> REST (DataPower)	Lambda Invoke to <code>data_power_mock</code>
Model used	<code>global.anthropic.claude-haiku-4-5-20251001-v1:0</code>	<code>global.anthropic.claude-haiku-4-5-20251001-v1:0</code>	<code>global.anthropic.claude-haiku-4-5-20251001-v1:0</code> when <code>USE_BEDROCK=0</code> otherwise deterministic (model)

	Claude AgentCore Runtime	Strands AgentCore Harness	LangGraph + Step Functions
AWS resources in PoC	IAM roles, mock Lambda, S3 object	Same as Claude	DynamoDB, Step Functions, 3 Lambdas, IAM, CloudWatch Logs

4. What AWS API calls happen during a run

Claude / Strands PoC run

1. s3:PutObject to <SHARED_S3_BUCKET>/openapi/mock-datapower.json
2. iam:CreateRole for gateway-service, harness-execution, datapower-mock
3. lambda:CreateFunction for nexa-*-datapower-mock
4. lambda:InvokeFunction when the harness calls the mock (or aws lambda invoke for testing)

LangGraph PoC run

1. dynamodb:CreateTable for nexa-langgraph-checkpoints
2. iam:CreateRole for Lambda and Step Functions
3. lambda:CreateFunction for intake, execute, datapower-mock
4. states:CreateStateMachine
5. At test time: states:StartExecution -> dynamodb:GetItem (wait for token) -> states:SendTaskSuccess -> states:DescribeExecution

5. How to manually see a run in AWS

Claude / Strands

```
aws lambda invoke --function-name nexa-claude-datapower-mock \
  --payload '{"tool": "extract_fields", "case_id": "CONTACTS-123"}' \
  --cli-binary-format raw-in-base64-out response.json
cat response.json
```

LangGraph

```
# 1. Start
aws stepfunctions start-execution \
  --state-machine-arn $(terraform output -raw state_machine_arn) \
  --input '{"case_id":"CASE-001","case_text":"Broker wants group contacts updated"}' \
  --name test-CASE-001

# 2. Wait for token
aws dynamodb get-item \
  --table-name $(terraform output -raw dynamodb_table) \
```

```

--key '{"case_id":{"S":"CASE-001"},"sk":{"S":"token"}}'

# 3. Resume
aws stepfunctions send-task-success \
  --task-token "<token-from-step-2>" \
  --output '{"case_id":"CASE-001","decision":"Approved"}'

# 4. Check result
aws stepfunctions describe-execution --execution-arn <arn-from-
step-1>

```

6. Notes from the test environment

- The target test account (ap-south-1) has a Service Control Policy that denies s3:CreateBucket for new nexa-* buckets, so the AgentCore scenarios use the existing <SHARED_S3_BUCKET> S3 bucket.
- A tag policy rejected Environment = "poc" and Environment = "dev" on Lambda, so the Terraform modules were updated to omit the Environment tag from all resources.
- The test_langgraph.py script requires boto3, which is not installed in the default Python; it should be run from a virtualenv with boto3 and botocore.
- **LangGraph Bedrock-enabled run:** The latest LangGraph run used USE_BEDROCK=true with global.anthropic.claude-haiku-4-5-20251001-v1:0. Bedrock selected the same 5-tool intake sequence (extract_fields → retrieve_sop → run_validations → search_similar_cases → create_decision_task), then the execution paused at waitForTaskToken, was resumed with Approved, and completed execute_transaction + write_audit_record → SUCCEEDED / status: CLOSED. An earlier deterministic run (USE_BEDROCK=false, CASE-REPORT-001) used the same tool sequence without invoking a model.
- **Model access constraints in ap-south-1:** anthropic.claude-3-5-sonnet-20241022-v2:0 requires a cross-region inference profile (e.g. apac.anthropic.claude-3-5-sonnet-20241022-v2:0) and the account must submit the Anthropic “model use case details” form for streaming Converse. global.anthropic.claude-haiku-4-5-20251001-v1:0 was already enabled and used for all three successful runs.
- **All AWS resources were destroyed after testing.** No NEXA gateways, harnesses, Lambda functions, IAM roles, Step Functions state machines, DynamoDB tables, or temporary S3 objects remain.